YARG Contributions — Roadmap

Canonical roadmap for both repos in this project (`yarg-song-server` and the `yarg` client fork). Status as of 2026-09-05.

Two tracks run in parallel. The **server track** is the #1 priority and is sequential — each phase depends on the one before it. The **client track** is independent and can absorb spare cycles at any time.

Phase 0 — Foundations ✅

Repos, remotes, mirrors and the format research that everything else is built on.

- [x] `fatalexception/yarg-song-server` and `fatalexception/yarg` created on Vault2 GitLab
- [x] Song-format research spec written (`docs/research/yarg-song-formats.md`)
- [x] Architecture decision recorded (`docs/ADR-001-server-architecture.md`)
- [x] Project instructions mirrored into `CLAUDE.md` so both machines pick them up
- [x] `yarg` imported from upstream server-side: 21 branches, 172 MB of repository and 180 MB of LFS objects, default branch set to `dev`
- [x] Public push mirrors to `github.com/Coffeehedake` configured and verified: `yarg-song-server` (GitLab mirror 10) and `yarg` (mirror 11), both one-way, all branches, divergent refs not kept. First sync 2026-09-05 16:44 ET, both `finished` with no error, and the GitHub side matches GitLab commit-for-commit.

Deliberately deferred: the `yarg` fork is *not* cloned locally. The project folder is inside the Syncthing `dev-projects` mesh, so a local clone replicates ~350 MB to r7 and Vault2 for a repo nothing touches until Phase 3. Clone it when Phase 3 starts:

```
cd "C:\dev\YARG - Open Source Contributions"
git clone https://gitlab.badassium.com/fatalexception/yarg.git yarg
cd yarg
git remote add upstream https://github.com/YARC-Official/YARG.git
git submodule update --init --recursive
git lfs pull
```

Exit criterion: both repos exist, both push to Vault2, both mirror to GitHub, and the format spec is committed. **Met 2026-09-05.**

`Coffeehedake/yarg` is a real GitHub *fork* of `YARC-Official/YARG`, not a plain repository, because that is the only shape GitHub accepts a pull request to upstream from.

Correction — LFS does propagate. An earlier revision of this roadmap claimed GitLab push mirroring drops LFS objects, and that the fork relationship was what made upstream's ~180 MB resolve. That was asserted from priors, not measured, and it is wrong.

The measurement: a throwaway GitLab project (**not** a fork, so no parent LFS storage to borrow from) with `*.png filter=lfs`, one fresh 1 MB object, push-mirrored to a throwaway **non-fork** GitHub repo. GitHub's LFS batch API returned a download action with no error, and the object downloaded from `github-cloud.githubusercontent.com` at exactly 1,048,576 bytes with a SHA-256 matching the OID. GitLab 18.11.3, git-lfs 3.7.1. Both throwaway repos were deleted afterwards.

So no special handling is needed when client work adds one of the five patterns YARG's `.gitattributes` tracks — `*.png`, `*.exr`, `*.jpg`, `*.fbx`, `*.ttf`. Which is just as well: almost any UI work in Phase 3 adds a `.png`.

The caveat that does survive: the mirror force-overwrites and the GitHub side is a fork, so anything done directly on GitHub is clobbered on the next sync. To open an upstream PR, create the branch on GitLab, let it mirror, open the PR from the mirrored branch, and leave it alone on GitHub after that.

Phase 1 — `yargsong`: the Go format library

The server is worthless until Go can read and write what YARG reads and writes. This phase is pure library work with no network surface, which makes it the easiest phase to test exhaustively.

Build order (each step is independently testable):

1. **`song.ini` reader** — ✅ done (`internal/songini`). ~130 recognised keys with their types, and a deliberately lenient reader: UTF-8/UTF-16 BOMs, Latin-1 fallback so accented artist names survive, `[song]` / `[Song]` /no-header variance, trailing junk after the section bracket, values containing `=`, and malformed lines skipped rather than fatal. Two behaviours were checked against upstream rather than guessed: **duplicate keys — last wins** (upstream stores modifiers by plain dictionary assignment), and **keys and section names are lowercased** before lookup. One thing is still assumed and flagged in the source: the exact whitespace and multiple-`=` handling inside `YARGTextReader.ExtractModifierName`, which has not been read. The writer is not built yet.

2. `.sng` **reader** — ✅ done (`internal/sng`). Implemented as an `fs.FS`, including synthesised directories, so the folder scanner and the `.sng` scanner share exactly one code path; `fstest.TestFS` enforces that. The mask-origin question the research doc flagged as unconfirmed is now settled from `SngFileStream.cs`: **the XOR index is the byte offset within each contained file, restarting at 0 per file**, not the absolute offset in the `.sng`. Upstream reaches the same result only because it decrypts whole 1 MB buffers and 1 MB is a multiple of the 256-byte table; tracking the real offset is equivalent and survives arbitrary read sizes. Malformed input is rejected with an error rather than a panic, which is tested.
3. **Scanner** — walks a folder or `.sng`, applies YARG's chart-file priority order (`notes.mid` → `notes.midi` → `notes.chart` → `notes.txt`), discovers stems/art/background, and emits the v1 catalog schema.
4. **Identity** — `SHA1(chart file bytes)`, matching YARG's `HashWrapper` exactly, so client and server agree on "do I already have this". Plus a `package_hash` of our own for same-chart-different-audio cases. Model hash → *many* entries; duplicates are normal.
5. `.sng` **writer** — validated two ways: round-trip through the reference `SngCli`, and by confirming a real YARG install scans the output and shows correct metadata.
6. **MIDI preparsers** — derive which instruments and difficulties actually exist. Only note-range walking is needed, not full chart semantics. This step is deliberately last; everything before it works with `parts_derived: false`.

Exit criterion: a CLI that scans a real song library, emits a catalog, repacks to `.sng`, and YARG scans the repacked output with identical metadata and an identical hash.

Explicitly out of scope, permanently: CON/mogg decryption, `songcache.bin` generation, `.milo_xbox` / `.png_xbox` decoding, `.yarground` inspection, full MIDI chart semantics.

Phase 2 — The server, and a sync client that needs no client changes

Two deliverables. The sync client is what makes the server *useful* before any client fork work exists, and it is the proof that the server is correct.

2a — `yarg-song-server`

- HTTP API: browse/search over the 12 attributes YARG itself sorts by, `GET /song/{hash}.sng`, and a bulk `POST /have` that takes a list of hashes and answers what the client is missing.
- Ingest: loose folders, `.sng`, and `.zip` / `.7z` of a loose folder. Refuse RB packages with a clear message.
- Config file + sane defaults; no database server required for v1 (embedded store).
- Docker image, multi-arch `linux/amd64` + `linux/arm64` (Pi), plus native macOS and Windows binaries from the same source. CI builds all of them.

2b — sync client

A small companion binary that pulls the server's library into an ordinary local songs folder. Unmodified YARG then just sees files. This ships real value in days, with zero risk to the client, and exercises the whole server end-to-end.

Exit criterion: a Pi on the LAN serves a shared library; two machines running stock YARG both see the same songs without either one being modified.

Phase 3 — Native remote song source in the client fork

Only now does the `yarg/` fork get touched. This is the upstream-facing work.

- Add an HTTP song source alongside local folders — browse, stream, cache.
- Touches YARG's song cache, so it must be designed with upstream in mind rather than bolted on.
- Upstream posture: `CONTRIBUTING.md` says nothing about networking or remote sources in any of its six tiers. That is an absence, not an endorsement — **open a discussion with YARC before building the PR**, and frame it as a content source, not a new game mode.
- PRs target `dev`. Never `master`.

Exit criterion: the fork can browse and play from a server without a sync step, and a discussion thread exists upstream.

Phase 4 — Modular features and the server config menu

The point at which the server stops being one feature and becomes a platform.

- Feature registry: each capability is opt-in and independently enable-able.
- Config UI in the server app, so a user turns on only what they want.
- Candidate modules: multi-user libraries and permissions, playlists/setlists shared across clients, scores and leaderboards, library health reporting.

Phase 5 — LLM chart generation

The long-term goal, and the phase most likely to move. It depends on every phase above working.

- Upload any song; auto-generate instrument and vocal parts across all difficulty levels.
- Produce a complete, working, **editable** package — generated charts are a starting point, not a final answer.

- **Distribution constraint, non-negotiable:** the chart/vocal tracks must be packageable and distributable *separately from the audio*, so charts can be shared without the music.
- Runs against the existing GPU arbitration on the home estate rather than a new stack.

Client track (independent of the server line)

Can be picked up at any time; does not block the server.

- **Controller compatibility** — official Rock Band and Guitar Hero instruments first. Upstream handles this through PlasticBand-Unity, so fixes may belong there rather than in YARG.
- **Graphics** — general rendering and visual improvements.
- Both are ordinary upstream contributions: fork, branch off `dev`, PR to `dev`.

Blockers

None. The Coffeehedake GitHub PAT was rotated on 2026-09-05 and verified (`login=Coffeehedake`, scopes `repo`, `workflow`, `project`, `write:packages`, `delete:packages`, `audit_log`).

Go 1.27.0 was installed on ENG-1 on 2026-09-05, so the toolchain claim is now measured on the machine the work actually happens on: `gofmt` clean, `go vet` exit 0, `go test ./...` green, and all six release targets building — linux/amd64, linux/arm64, linux/armv7, darwin/amd64, darwin/arm64, windows/amd64.

It is a **per-user** install at `%LOCALAPPDATA%\Programs\go`, from the official zip with its SHA-256 verified against `go.dev/dl/?mode=json` before extraction, with `go\bin` and `%USERPROFILE%\go\bin` added to the user `Path`. Per-user rather than machine-wide on purpose: the MSI needs elevation, and an unattended `msiexec /qn` from a non-elevated session fails *silently* — the same failure mode that produced the Bambu Studio update loop. Nothing about this install needs elevation, and `GOPATH` / `GOCACHE` land in the user profile rather than in the Syncthing root. Promote it to a machine-wide install later if you want; nothing depends on where it lives.

Sequencing note

Phases 1 and 2 are the whole of the "#1 priority". Nothing in phases 3–5 should start before a stock YARG client is reading songs from a self-hosted server, because every later decision depends on what that turns out to actually require.
